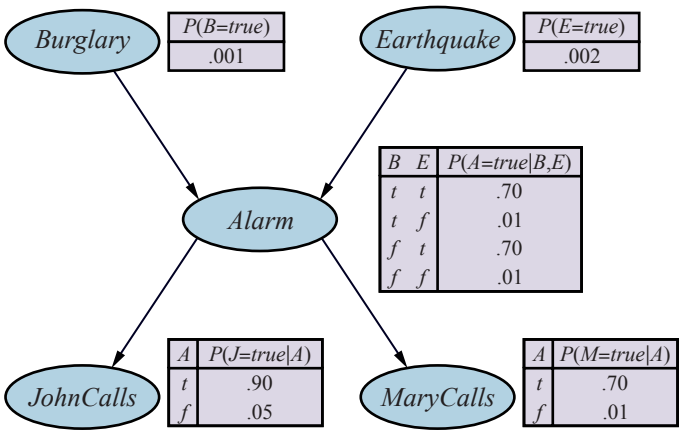


Figure 13.2 A typical Bayesian network, showing both the topology and the conditional probability tables (CPTs). In the CPTs, the letters B, E, A, J, and M stand for *Burglary*, *Earthquake*, *Alarm*, *JohnCalls*, and *MaryCalls*, respectively.

Conditioning case

tion 13.2.) Each row in a CPT contains the conditional probability of each node value for a **conditioning case**. A conditioning case is just a possible combination of values for the parent nodes—a miniature possible world, if you like. Each row must sum to 1, because the entries represent an exhaustive set of cases for the variable. For Boolean variables, once you know that the probability of a true value is p, the probability of false must be $1-p$, so we often omit the second number, as in Figure 13.2. In general, a table for a Boolean variable with k Boolean parents contains 2^k independently specifiable probabilities. A node with no parents has only one row, representing the prior probabilities of each possible value of the variable.

Notice that the network does not have nodes corresponding to Mary's currently listening to loud music or to the telephone ringing and confusing John. These factors are summarized in the uncertainty associated with the links from *Alarm* to *JohnCalls* and *MaryCalls*. This shows both laziness and ignorance in operation, as explained on page 386: it would be a lot of work to find out why those factors would be more or less likely in any particular case, and we have no reasonable way to obtain the relevant information anyway.

The probabilities actually summarize a *potentially infinite* set of circumstances in which the alarm might fail to go off (high humidity, power failure, dead battery, cut wires, a dead mouse stuck inside the bell, etc.) or John or Mary might fail to call and report it (out to lunch, on vacation, temporarily deaf, passing helicopter, etc.). In this way, a small agent can cope with a very large world, at least approximately.

13.2 The Semantics of Bayesian Networks

The *syntax* of a Bayes net consists of a directed acyclic graph with some local probability information attached to each node. The *semantics* defines how the syntax corresponds to a joint distribution over the variables of the network.

Assume that the Bayes net contains n variables, $X_1,\ldots,X_n$. A generic entry in the joint distribution is then $P(X_1=x_1\wedge\ldots\wedge X_n=x_n)$, or $P(x_1,\ldots,x_n)$ for short. The semantics of

Bayes nets defines each entry in the joint distribution as follows:

$$P(x_1,\ldots,x_n) = \prod_{i=1}^{n} \theta(x_i \mid parents(X_i)), \tag{13.1}$$

where $parents(X_i)$ denotes the values of $Parents(X_i)$ that appear in $x_1,\ldots,x_n$. Thus, each entry in the joint distribution is represented by the product of the appropriate elements of the local conditional distributions in the Bayes net.

To illustrate this, we can calculate the probability that the alarm has sounded, but neither a burglary nor an earthquake has occurred, and both John and Mary call. We simply multiply the relevant entries from the local conditional distributions (abbreviating the variable names):

$$\begin{aligned} P(j,m,a,\neg b,\neg e) &= P(j \mid a)P(m \mid a)P(a \mid \neg b \wedge \neg e)P(\neg b)P(\neg e) \\ &= 0.90 \times 0.70 \times 0.01 \times 0.999 \times 0.998 = 0.00628\,. \end{aligned}$$

Section 12.3 explained that the full joint distribution can be used to answer any query about the domain. If a Bayes net is a representation of the joint distribution, then it too can be used to answer any query, by summing all the relevant joint probability values, each calculated by multiplying probabilities from the local conditional distributions. Section 13.3 explains this in more detail, but also describes methods that are much more efficient.

So far, we have glossed over one important point: what is the meaning of the numbers that go into the local conditional distributions $\theta(x_i \mid parents(X_i))$? It turns out that from Equation (13.1) we can prove that the parameters $\theta(x_i \mid parents(X_i))$ are exactly the conditional probabilities $P(x_i \mid parents(X_i))$ implied by the joint distribution. Remember that the conditional probabilities can be computed from the joint distribution as follows:

$$\begin{aligned} P(x_i \mid parents(X_i)) &\equiv \frac{P(x_i, parents(X_i))}{P(parents(X_i))} \\ &= \frac{\sum_{\mathbf{y}} P(x_i, parents(X_i), \mathbf{y})}{\sum_{x_i', \mathbf{y}} P(x_i', parents(X_i), \mathbf{y})} \end{aligned}$$

where $\mathbf{y}$ represents the values of all variables other than X_i and its parents. From this last line one can prove that $P(x_i \mid parents(X_i)) = \theta(x_i \mid parents(X_i))$ (Exercise 13.CPTE). Hence, we can rewrite Equation (13.1) as

$$P(x_1,\ldots,x_n) = \prod_{i=1}^{n} P(x_i \mid parents(X_i)). \tag{13.2}$$

This means that when one estimates values for the local conditional distributions, they need to be the actual conditional probabilities for the variable given its parents. So, for example, when we specify $\theta(JohnCalls = true \mid Alarm = true) = 0.90$, it should be the case that about 90% of the time when the alarm sounds, John calls. The fact that each parameter of the network has a precise meaning in terms of only a small set of variables is crucially important for robustness and ease of specification of the models.

A method for constructing Bayesian networks

Equation (13.2) defines what a given Bayes net means. The next step is to explain how to *construct* a Bayesian network in such a way that the resulting joint distribution is a good representation of a given domain. We will now show that Equation (13.2) implies certain conditional independence relationships that can be used to guide the knowledge engineer in

constructing the topology of the network. First, we rewrite the entries in the joint distribution in terms of conditional probability, using the product rule (see page 390):

$$P(x_1,\ldots,x_n) = P(x_n \,|\, x_{n-1},\ldots,x_1)P(x_{n-1},\ldots,x_1)\,.$$

Then we repeat the process, reducing each joint probability to a conditional probability and a joint probability on a smaller set of variables. We end up with one big product:

$$\begin{aligned} P(x_1,\ldots,x_n) &= P(x_n \,|\, x_{n-1},\ldots,x_1)P(x_{n-1} \,|\, x_{n-2},\ldots,x_1) \cdots P(x_2 \,|\, x_1)P(x_1) \\ &= \prod_{i=1}^{n} P(x_i \,|\, x_{i-1},\ldots,x_1)\,. \end{aligned}$$

Chain rule

This identity is called the **chain rule**. It holds for any set of random variables. Comparing it with Equation (13.2), we see that the specification of the joint distribution is equivalent to the general assertion that, for every variable X_i in the network,

$$\mathbf{P}(X_i \,|\, X_{i-1},\ldots,X_1) = \mathbf{P}(X_i \,|\, Parents(X_i))\,, \tag{13.3}$$

Topological ordering

provided that $Parents(X_i) \subseteq \{X_{i-1},\ldots,X_1\}$. This last condition is satisfied by numbering the nodes in **topological order**—that is, in any order consistent with the directed graph structure. For example, the nodes in Figure 13.2 could be ordered B,E,A,J,M; E,B,A,M,J; and so on.

What Equation (13.3) says is that the Bayesian network is a correct representation of the domain only if each node is conditionally independent of its other predecessors in the node ordering, given its parents. We can satisfy this condition with this methodology:

1. *Nodes:* First determine the set of variables that are required to model the domain. Now order them, $\{X_1,\ldots,X_n\}$. Any order will work, but the resulting network will be more compact if the variables are ordered such that causes precede effects.
2. *Links:* For $i = 1$ to n do:
   - Choose a minimal set of parents for X_i from $X_1,\ldots,X_{i-1}$, such that Equation (13.3) is satisfied.
   - For each parent insert a link from the parent to X_i.
   - CPTs: Write down the conditional probability table, $\mathbf{P}(X_i|Parents(X_i))$.

Intuitively, the parents of node X_i should contain all those nodes in $X_1,\ldots,X_{i-1}$ that *directly influence* X_i. For example, suppose we have completed the network in Figure 13.2 except for the choice of parents for *MaryCalls*. *MaryCalls* is certainly influenced by whether there is a *Burglary* or an *Earthquake*, but not *directly* influenced. Intuitively, our knowledge of the domain tells us that these events influence Mary's calling behavior only through their effect on the alarm. Also, given the state of the alarm, whether John calls has no influence on Mary's calling. Formally speaking, we believe that the following conditional independence statement holds:

$$\mathbf{P}(MaryCalls \,|\, JohnCalls, Alarm, Earthquake, Burglary) = \mathbf{P}(MaryCalls \,|\, Alarm)\,.$$

Thus, *Alarm* will be the only parent node for *MaryCalls*.

Because each node is connected only to earlier nodes, this construction method guarantees that the network is acyclic. Another important property of Bayes nets is that they contain no redundant probability values. If there is no redundancy, then there is no chance for inconsistency: *it is impossible for the knowledge engineer or domain expert to create a Bayesian network that violates the axioms of probability.*

Compactness and node ordering

As well as being a complete and nonredundant representation of the domain, a Bayes net can often be far more *compact* than the full joint distribution. This property is what makes it feasible to handle domains with many variables. The compactness of Bayesian networks is an example of a general property of **locally structured** (also called **sparse**) systems. In a locally structured system, each subcomponent interacts directly with only a bounded number of other components, regardless of the total number of components. Local structure is usually associated with linear rather than exponential growth in complexity.

Locally structured
Sparse

In the case of Bayes nets, it is reasonable to suppose that in most domains each random variable is directly influenced by at most k others, for some constant k. If we assume n Boolean variables for simplicity, then the amount of information needed to specify each conditional probability table will be at most 2^k numbers, and the complete network can be specified by $2^k \cdot n$ numbers. In contrast, the joint distribution contains 2^n numbers. To make this concrete, suppose we have $n = 30$ nodes, each with five parents ($k = 5$). Then the Bayesian network requires 960 numbers, but the full joint distribution requires over a billion.

Specifying the conditional probability tables for a fully connected network, in which each variable has all of its predecessors as parents, requires the same amount of information as specifying the joint distribution in tabular form. For this reason, we often leave out links even though a slight dependency exists, because the slight gain in accuracy is not worth the the additional complexity in the network. For example, one might object to our burglary network on the grounds that if there is a large earthquake, then John and Mary would not call even if they heard the alarm, because they assume that the earthquake is the cause. Whether to add the link from *Earthquake* to *JohnCalls* and *MaryCalls* (and thus enlarge the tables) depends on the importance of getting more accurate probabilities compared with the cost of specifying the extra information.

Even in a locally structured domain, we will get a compact Bayes net only if we choose the node ordering well. What happens if we happen to choose the wrong order? Consider the burglary example again. Suppose we decide to add the nodes in the order *MaryCalls*, *JohnCalls*, *Alarm*, *Burglary*, *Earthquake*. We then get the somewhat more complicated network shown in Figure 13.3(a). The process goes as follows:

- Adding *MaryCalls*: No parents.
- Adding *JohnCalls*: If Mary calls, that probably means the alarm has gone off, which makes it more likely that John calls. Therefore, *JohnCalls* needs *MaryCalls* as a parent.
- Adding *Alarm*: Clearly, if both call, it is more likely that the alarm has gone off than if just one or neither calls, so we need both *MaryCalls* and *JohnCalls* as parents.
- Adding *Burglary*: If we know the alarm state, then the call from John or Mary might give us information about our phone ringing or Mary's music, but not about burglary:

  $$\mathbf{P}(Burglary \mid Alarm, JohnCalls, MaryCalls) = \mathbf{P}(Burglary \mid Alarm)\,.$$

  Hence we need just *Alarm* as parent.
- Adding *Earthquake*: If the alarm is on, it is more likely that there has been an earthquake. (The alarm is an earthquake detector of sorts.) But if we know that there has been a burglary, then that explains the alarm, and the probability of an earthquake would be only slightly above normal. Hence, we need both *Alarm* and *Burglary* as parents.

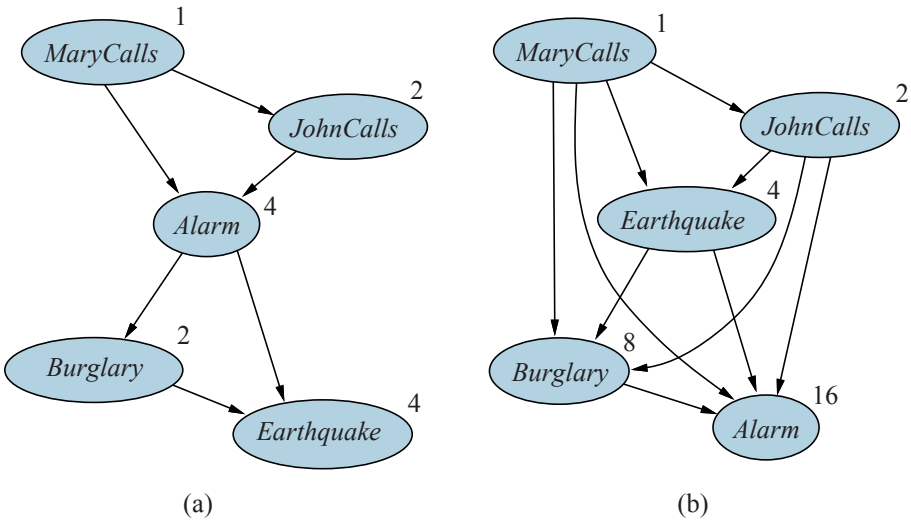


Figure 13.3 Network structure and number of parameters depends on order of introduction. (a) The structure obtained with ordering M, J, A, B, E. (b) The structure obtained with M, J, E, B, A. Each node is annotated with the number of parameters required; 13 in all for (a) and 31 for (b). In Figure 13.2, only 10 parameters were required.

The resulting network has two more links than the original network in Figure 13.2 and requires 13 conditional probabilities rather than 10. What's worse, some of the links represent tenuous relationships that require difficult and unnatural probability judgments, such as assessing the probability of *Earthquake*, given *Burglary* and *Alarm*. This phenomenon is quite general and is related to the distinction between **causal** and **diagnostic** models introduced in Section 12.5.1 (see also Exercise 13.WUMD). *If we stick to a causal model, we end up having to specify fewer numbers, and the numbers will often be easier to come up with.* For example, in the domain of medicine, it has been shown by Tversky and Kahneman (1982) that expert physicians prefer to give probability judgments for causal rules rather than for diagnostic ones. Section 13.5 explores the idea of causal models in more depth.

Figure 13.3(b) shows a very bad node ordering: *MaryCalls*, *JohnCalls*, *Earthquake*, *Burglary*, *Alarm*. This network requires 31 distinct probabilities to be specified—exactly the same number as the full joint distribution. It is important to realize, however, that any of the three networks can represent *exactly the same joint distribution*. The two versions in Figure 13.3 simply fail to represent all the conditional independence relationships and hence end up specifying a lot of unnecessary numbers instead.

13.2.1 Conditional independence relations in Bayesian networks

From the semantics of Bayes nets as defined in Equation (13.2), we can derive a number of conditional independence properties. We have already seen the property that a variable is conditionally independent of its other predecessors, given its parents. It is also possible to prove the more general "non-descendants" property that:

Descendant

*Each variable is conditionally independent of its non-**descendants**, given its parents.*

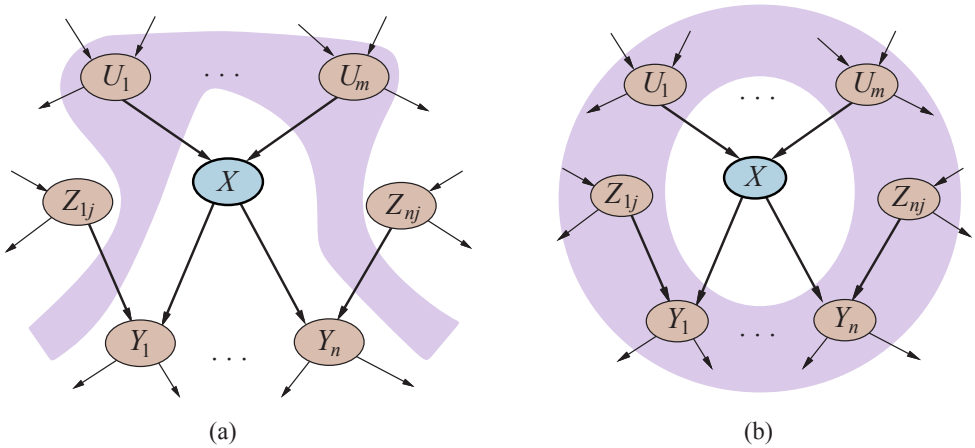


Figure 13.4 (a) A node X is conditionally independent of its non-descendants (e.g., the Z_{ij}s) given its parents (the U_is shown in the gray area). (b) A node X is conditionally independent of all other nodes in the network given its Markov blanket (the gray area).

For example, in Figure 13.2, the variable *JohnCalls* is independent of *Burglary*, *Earthquake*, and *MaryCalls* given the value of *Alarm*. The definition is illustrated in Figure 13.4(a).

It turns out that the non-descendants property combined with interpretation of the network parameters $\theta(X_i \mid Parents(X_i))$ as conditional probabilities $\mathbf{P}(X_i \mid Parents(X_i))$ suffices to reconstruct the full joint distribution given in Equation (13.2). In other words, one can view the semantics of Bayes nets in a different way: instead of defining the full joint distribution as the product of conditional distributions, the network defines a set of conditional independence properties. The full joint distribution can be derived from those properties.

Another important independence property is implied by the non-descendants property:

> a variable is conditionally independent of all other nodes in the network, given its parents, children, and children's parents—that is, given its **Markov blanket**.

Markov blanket

(Exercise 13.MARB asks you to prove this.) For example, the variable *Burglary* is independent of *JohnCalls* and *MaryCalls*, given *Alarm* and *Earthquake*. This property is illustrated in Figure 13.4(b). The Markov blanket property makes possible inference algorithms that use completely local and distributed stochastic sampling processes, as explained in Section 13.4.2.

The most general conditional independence question one might ask in a Bayes net is whether a set of nodes $\mathbf{X}$ is conditionally independent of another set $\mathbf{Y}$, given a third set $\mathbf{Z}$. This can be determined efficiently by examining the Bayes net to see whether $\mathbf{Z}$ **d-separates** $\mathbf{X}$ and $\mathbf{Y}$. The process works as follows:

D-separation

1. Consider just the **ancestral subgraph** consisting of $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{Z}$, and their ancestors. Ancestral subgraph
2. Add links between any unlinked pair of nodes that share a common child; now we have the so-called **moral graph**. Moral graph
3. Replace all directed links by undirected links.
4. If $\mathbf{Z}$ blocks all paths between $\mathbf{X}$ and $\mathbf{Y}$ in the resulting graph, then $\mathbf{Z}$ d-separates $\mathbf{X}$ and $\mathbf{Y}$. In that case, $\mathbf{X}$ is conditionally independent of $\mathbf{Y}$, given $\mathbf{Z}$. Otherwise, the original Bayes net does not require conditional independence.

In brief, then, d-separation means separation in the undirected, moralized, ancestral subgraph. Applying the definition to the burglary network in Figure 13.2, we can deduce that *Burglary* and *Earthquake* are independent given the empty set (i.e., they are absolutely independent); that they are *not* necessarily conditionally independent given *Alarm*; and that *JohnCalls* and *MaryCalls* are conditionally independent given *Alarm*. Notice also that the Markov blanket property follows directly from the d-separation property, since a variable's Markov blanket d-separates it from all other variables.

13.2.2 Efficient Representation of Conditional Distributions

Even if the maximum number of parents k is smallish, filling in the CPT for a node requires up to $O(2^k)$ numbers and perhaps a great deal of experience with all the possible conditioning cases. In fact, this is a worst-case scenario in which the relationship between the parents and the child is completely arbitrary. Usually, such relationships are describable by a **canonical distribution** that fits some standard pattern. In such cases, the complete table can be specified just by naming the pattern and perhaps supplying a few parameters.

Canonical distribution

Deterministic nodes

The simplest example is provided by **deterministic nodes**. A deterministic node has its value specified exactly by the values of its parents, with no uncertainty. The relationship can be a logical one: for example, the relationship between the parent nodes *Canadian*, *US*, *Mexican* and the child node *NorthAmerican* is simply that the child is the disjunction of the parents. The relationship can also be numerical: for example, the *BestPrice* for a car is the minimum of the prices at each dealer in the area; and the *WaterStored* in a reservoir at year's end is the sum of the original amount, plus the inflows (rivers, runoff, precipitation) and minus the outflows (releases, evaporation, seepage).

Many Bayes net systems allow the user to specify deterministic functions using a general-purpose programming language; this makes it possible to include complex elements such as global climate models or power-grid simulators within a probabilistic model.

Context-specific independence

Another important pattern that occurs often in practice is **context-specific independence** or CSI. A conditional distribution exhibits CSI if a variable is conditionally independent of some of its parents given *certain values* of others. For example, let's suppose that the *Damage* to your car occurring during a given period of time depends on the *Ruggedness* of your car and whether or not an *Accident* occurred in that period. Clearly, if *Accident* is false, then the *Damage*, if any, doesn't depend on the *Ruggedness* of your car. (There might be vandalism damage to the car's paintwork or windows, but we'll assume all cars are equally subject to such damage.) We say that *Damage* is context-specifically independent of *Ruggedness* given $Accident = false$. Bayes net systems often implement CSI using an if-then-else syntax for specifying conditional distributions; for example, one might write

$$\mathbf{P}(Damage \mid Ruggedness, Accident) = \\ \textbf{if } (Accident = false) \textbf{ then } d_1 \textbf{ else } d_2(Ruggedness)$$

where d_1 and d_2 represent arbitrary distributions. As with determinism, the presence of CSI in a network may facilitate efficient inference. All of the exact inference algorithms mentioned in Section 13.3 can be modified to take advantage of CSI to speed up computation.

Noisy-OR

Uncertain relationships can often be characterized by so-called **noisy** logical relationships. The standard example is the **noisy-OR** relation, which is a generalization of the logical OR. In propositional logic, we might say that *Fever* is true if and only if *Cold*, *Flu*, or *Malaria*

Cold	*Flu*	*Malaria*	$P(fever \mid \cdot)$	$P(\neg fever \mid \cdot)$
f	f	f	0.0	1.0
f	f	t	0.9	**0.1**
f	t	f	0.8	**0.2**
f	t	t	0.98	$0.02 = 0.2 \times 0.1$
t	f	f	0.4	**0.6**
t	f	t	0.94	$0.06 = 0.6 \times 0.1$
t	t	f	0.88	$0.12 = 0.6 \times 0.2$
t	t	t	0.988	$0.012 = 0.6 \times 0.2 \times 0.1$

Figure 13.5 A complete conditional probability table for $\P(Fever \mid Cold, Flu, Malaria)$, assuming a noisy-OR model with the the three q-values shown in bold.

are true. The noisy-OR model allows for uncertainty about the ability of each parent to cause the child to be true—the causal relationship between parent and child may be *inhibited*, and so a patient could have a cold, but not exhibit a fever.

The model makes two assumptions. First, it assumes that all the possible causes are listed. (If some are missing, we can always add a so-called **leak node** that covers "miscellaneous causes.") Second, it assumes that inhibition of each parent is independent of inhibition of any other parents: for example, whatever inhibits *Malaria* from causing a fever is independent of whatever inhibits *Flu* from causing a fever. Given these assumptions, *Fever* is *false* if and only if all its *true* parents are inhibited, and the probability of this is the product of the inhibition probabilities q_j for each parent. Let us suppose these individual inhibition probabilities are as follows:

Leak node

$$q_{\text{cold}} = P(\neg fever \mid cold, \neg flu, \neg malaria) = 0.6\,,$$
$$q_{\text{flu}} = P(\neg fever \mid \neg cold, flu, \neg malaria) = 0.2\,,$$
$$q_{\text{malaria}} = P(\neg fever \mid \neg cold, \neg flu, malaria) = 0.1\,.$$

Then, from this information and the noisy-OR assumptions, the entire CPT can be built. The general rule is that

$$P(x_i \mid parents(X_i)) = 1 - \prod_{\{j : X_j = true\}} q_j\,,$$

where the product is taken over the parents that are set to true for that row of the CPT. Figure 13.5 illustrates this calculation.

In general, noisy logical relationships in which a variable depends on k parents can be described using $O(k)$ parameters instead of $O(2^k)$ for the full conditional probability table. This makes assessment and learning much easier. For example, the CPCS network (Pradhan *et al.*, 1994) uses noisy-OR and noisy-MAX distributions to model relationships among diseases and symptoms in internal medicine. With 448 nodes and 906 links, it requires only 8,254 parameters instead of 133,931,430 for a network with full CPTs.

13.2.3 Bayesian nets with continuous variables

Many real-world problems involve continuous quantities, such as height, mass, temperature, and money. By definition, continuous variables have an infinite number of possible values, so it is impossible to specify conditional probabilities explicitly for each value. One way to handle continuous variables is with **discretization**—that is, dividing up the possible values into a fixed set of intervals. For example, temperatures could be divided into three categories: ($<0^{\circ}$C), (0°C-100°C), and ($>100^{\circ}$C). In choosing the number of categories, there is a tradeoff between loss of accuracy and large CPTs which can lead to slow run times.

Discretization

Another approach is to define a continuous variable using one of the standard families of probability density functions (see Appendix A). For example, a Gaussian (or normal) distribution $\mathcal{N}(x;\mu,\sigma^2)$ is specified by just two parameters, the mean μ and the variance σ^2. Yet another solution—sometimes called a **nonparametric** representation—is to define the conditional distribution implicitly with a collection of instances, each containing specific values of the parent and child variables. We explore this approach further in Chapter 19.

Nonparametric

A network with both discrete and continuous variables is called a **hybrid Bayesian network**. To specify a hybrid network, we have to specify two new kinds of distributions: the conditional distribution for a continuous variable given discrete or continuous parents; and the conditional distribution for a discrete variable given continuous parents. Consider the simple example in Figure 13.6, in which a customer buys some fruit depending on its cost, which depends in turn on the size of the harvest and whether the government's subsidy scheme is operating. The variable *Cost* is continuous and has continuous and discrete parents; the variable *Buys* is discrete and has a continuous parent.

Hybrid Bayesian network

For the *Cost* variable, we need to specify $\mathbf{P}(Cost\,|\,Harvest, Subsidy)$. The discrete parent is handled by enumeration—that is, by specifying both $\mathbf{P}(Cost\,|\,Harvest, subsidy)$ and $\mathbf{P}(Cost\,|\,Harvest, \neg subsidy)$. To handle *Harvest*, we specify how the distribution over the cost c depends on the continuous value h of *Harvest*. In other words, we specify the *parameters* of the cost distribution as a function of h. The most common choice is the **linear–Gaussian** conditional distribution, in which the child has a Gaussian distribution whose mean μ varies linearly with the value of the parent and whose standard deviation σ is fixed. We need two distributions, one for *subsidy* and one for $\neg subsidy$, with different parameters:

Linear–Gaussian

$$P(c\,|\,h, subsidy) = \mathcal{N}(c; a_t h + b_t, \sigma_t^2) = \frac{1}{\sigma_t\sqrt{2\pi}}\, e^{-\frac{1}{2}\left(\frac{c-(a_t h+b_t)}{\sigma_t}\right)^2}$$

$$P(c\,|\,h, \neg subsidy) = \mathcal{N}(c; a_f h + b_f, \sigma_f^2) = \frac{1}{\sigma_f\sqrt{2\pi}}\, e^{-\frac{1}{2}\left(\frac{c-(a_f h+b_f)}{\sigma_f}\right)^2}.$$

For this example, then, the conditional distribution for *Cost* is specified by naming the linear–Gaussian distribution and providing the parameters a_t, b_t, σ_t, a_f, b_f, and σ_f. Figures 13.7(a) and (b) show these two relationships. Notice that in each case the slope of c versus h is negative, because cost decreases as the harvest size increases. (Of course, the assumption of linearity implies that the cost becomes negative at some point; the linear model is reasonable only if the harvest size is limited to a narrow range.) Figure 13.7(c) shows the distribution $P(c\,|\,h)$, averaging over the two possible values of *Subsidy* and assuming that each has prior probability 0.5. This shows that even with very simple models, quite interesting distributions can be represented.

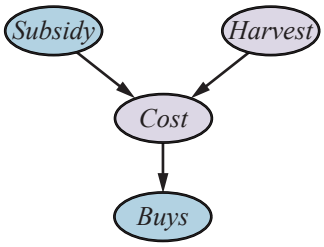
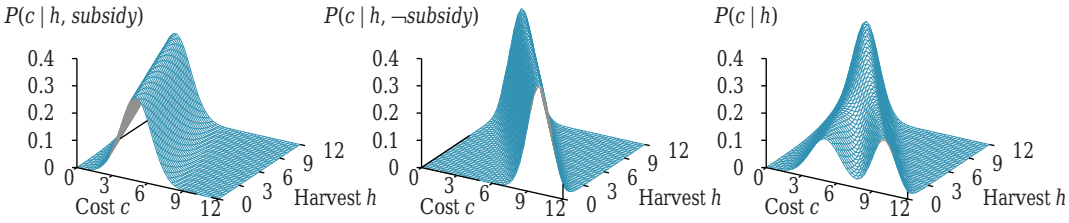


Figure 13.6 A simple network with discrete variables (*Subsidy* and *Buys*) and continuous variables (*Harvest* and *Cost*).



(a) (b) (c)

Figure 13.7 The graphs in (a) and (b) show the probability distribution over *Cost* as a function of *Harvest* size, with *Subsidy* true and false, respectively. Graph (c) shows the distribution $P(Cost \mid Harvest)$, obtained by summing over the two subsidy cases.

The linear–Gaussian conditional distribution has some special properties. A network containing only continuous variables with linear–Gaussian distributions has a joint distribution that is a multivariate Gaussian distribution (see Appendix A) over all the variables (Exercise 13.LGEX). Furthermore, the posterior distribution given any evidence also has this property.[2] When discrete variables are added as parents (not as children) of continuous variables, the network defines a **conditional Gaussian**, or CG, distribution: given any assignment to the discrete variables, the distribution over the continuous variables is a multivariate Gaussian. Conditional Gaussian

Now we turn to the distributions for discrete variables with continuous parents. Consider, for example, the *Buys* node in Figure 13.6. It seems reasonable to assume that the customer will buy if the cost is low and will not buy if it is high and that the probability of buying varies smoothly in some intermediate region. In other words, the conditional distribution is like a "soft" threshold function. One way to make soft thresholds is to use the *integral* of the standard normal distribution:

$$\Phi(x) = \int_{-\infty}^{x} \mathcal{N}(s;0,1)ds\,.$$

[2] It follows that inference in linear–Gaussian networks takes only $O(n^3)$ time in the worst case, regardless of the network topology. In Section 13.3, we see that inference for networks of discrete variables is NP-hard.

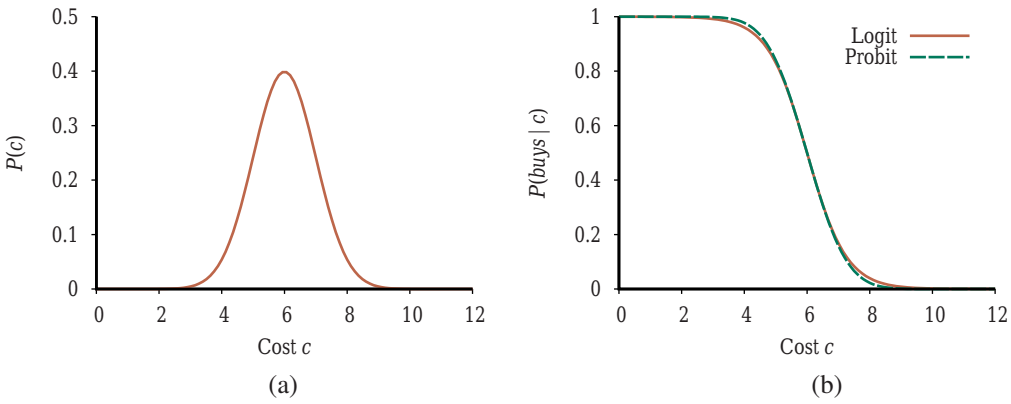


Figure 13.8 (a) A normal (Gaussian) distribution for the cost threshold, centered on $\mu = 6.0$ with standard deviation $\sigma = 1.0$. (b) Expit and probit models for the probability of *buys* given *cost*, for the parameters $\mu = 6.0$ and $\sigma = 1.0$.

$\Phi(x)$ is an increasing function of x, whereas the probability of buying decreases with cost, so here we flip the function around:

$$P(buys \mid Cost = c) = 1 - \Phi((c - \mu)/\sigma)\,,$$

Probit

which means that the cost threshold occurs around μ, the width of the threshold region is proportional to σ, and the probability of buying decreases as cost increases. This **probit** model (pronounced "pro-bit" and short for "probability unit") is illustrated in Figure 13.8(a). The form can be justified by proposing that the underlying decision process has a hard threshold, but that the precise location of the threshold is subject to random Gaussian noise.

Expit
Inverse logit
Logistic function

An alternative to the probit model is the **expit** or **inverse logit** model. It uses the **logistic function** $1/(1 + e^{-x})$ to produce a soft threshold—it maps any x to a value between 0 and 1. Again, for our example, we flip it around to make a decreasing function; we also scale the exponent by $4/\sqrt{2\pi}$ to match the probit's slope at the mean:

$$P(buys \mid Cost = c) = 1 - \frac{1}{1 + exp(-\frac{4}{\sqrt{2\pi}} . \frac{c-\mu}{\sigma})}\,.$$

This is illustrated in Figure 13.8(b). The two distributions look similar, but the logit actually has much longer "tails." The probit is often a better fit to real situations, but the logistic function is sometimes easier to deal with mathematically. It is used widely in machine learning. Both models can be generalized to handle multiple continuous parents by taking a linear combination of the parent values. This also works for discrete parents if their values are integers; for example, with k Boolean parents, each viewed as having values 0 or 1, the input to the expit or probit distribution would be a weighted linear combination with k parameters, yielding a model quite similar to the noisy-OR model discussed earlier.

13.2.4 Case study: Car insurance

A car insurance company receives an application from an individual to insure a specific vehicle and must decide on the appropriate annual premium to charge, based on the anticipated claims it will pay out for this applicant. The task is to build a Bayes net that captures the

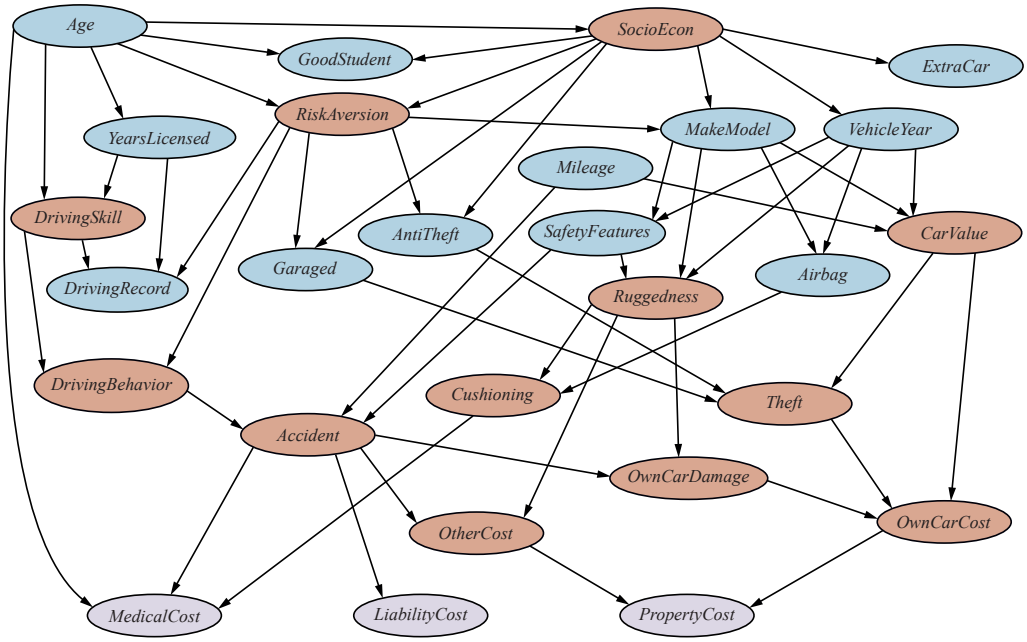


Figure 13.9 A Bayesian network for evaluating car insurance applications.

causal structure of the domain and gives an accurate, well-calibrated distribution over the output variables given the evidence available from the application form.[3] The Bayes net will include **hidden variables** that are neither input nor output variables, but are essential for structuring the network so that it is reasonably sparse with a manageable number of parameters. The hidden variables are shaded brown in Figure 13.9.

Hidden variable

The claims to be paid out—shaded lavender in Figure 13.9—are of three kinds: the *MedicalCost* for any injuries sustained by the applicant; the *LiabilityCost* for lawsuits filed by other parties against the applicant and the company; and the *PropertyCost* for vehicle damage to either party and vehicle loss by theft. The application form asks for the following input information (the light blue nodes in Figure 13.9):

- About the applicant: *Age*; *YearsLicensed*—how long since a driving license was first obtained; *DrivingRecord*—some summary, perhaps based on "points," of recent accidents and traffic violations; and (for students) a *GoodStudent* indicator for a grade-point average of 3.0 (B) on a 4-point scale.
- About the vehicle: the *MakeModel* and *VehicleYear*; whether it has an *Airbag*; and some summary of *SafetyFeatures* such as anti-lock braking and collision warning.
- About the driving situation: the annual *Mileage* driven and how securely the vehicle is *Garaged*, if at all.

[3] The network shown in Figure 13.9 is not in actual use, but its structure has been vetted with insurance experts. In practice, the information requested on application forms varies by company and jurisdiction—for example, some ask for *Gender*—and the model could certainly be made more detailed and sophisticated.

Now we need to think about how to arrange these into a causal structure. The key hidden variables are whether or not a *Theft* or *Accident* will occur in the next time period. Obviously, one cannot ask the applicant to predict these; they have to inferred from the available information and the insurer's previous experience.

What are the causal factors leading to *Theft*? The *MakeModel* is certainly important—some models are stolen much more often than others because there is an efficient resale market for vehicles and parts; the *CarValue* also matters, because an old, beat-up, or high-mileage vehicle has lower resale value. Moreover, a vehicle that is *Garaged* and has an *AntiTheft* device is harder to steal. The hidden variable *CarValue* depends in turn on the *MakeModel*, *VehicleYear*, and *Mileage*. *CarValue* also dictates the loss amount when a *Theft* occurs, so that is one of the contributors to *OwnCarCost* (the other being accidents, which we will get to shortly).

It is common in models of this type to introduce another hidden variable, *SocioEcon*, the socioeconomic category of the applicant. This is thought to influence a wide range of behaviors and characteristics. In our model, there is no *direct* evidence in the form of observed income and occupation variables;[4] but *SocioEcon* influences *MakeModel* and *VehicleYear*; it also affects *ExtraCar* and *GoodStudent*, and depends somewhat on *Age*.

For any insurance company, perhaps the most important hidden variable is *RiskAversion*: people who are risk-averse are good insurance risks! *Age* and *SocioEcon* affect *RiskAversion*, and its "symptoms" include the applicant's choice of whether the vehicle is *Garaged* and has *AntiTheft* devices and *SafetyFeatures*.

In predicting future accidents, the key is the applicant's future *DrivingBehavior*, which is influenced by both *RiskAversion* and *DrivingSkill*; the latter in turn depends on *Age* and *YearsLicensed*. The applicant's past driving behavior is reflected in the *DrivingRecord*, which also depends on *RiskAversion* and *DrivingSkill* as well as on *YearsLicensed* (because someone who started driving only recently may not have had time to accumulate a litany of accidents and violations). In this way, *DrivingRecord* provides evidence about *RiskAversion* and *DrivingSkill*, which in turn help to predict future *DrivingBehavior*.

We can think of *DrivingBehavior* as a per-mile tendency to drive in an accident-prone way; whether an *Accident* actually occurs in a fixed time period depends also on the annual *Mileage* and on the *SafetyFeatures* of the vehicle. If an *Accident* occurs, there are three kinds of costs: the *MedicalCost* for the applicant depends on *Age* and *Cushioning*, which depends in turn on the *Ruggedness* of the car and whether it has an *Airbag*; the *LiabilityCost* (medical, pain and suffering, loss of income, etc.) for the other driver; and the *PropertyCost* for the applicant and the other driver, both of which depend (in different ways) on the car's *Ruggedness* and on the applicant's *CarValue*.

We have illustrated the kind of reasoning that goes into developing the topology and hidden variables in a Bayes net. We also need to specify the ranges and the conditional distributions for each variable. For the ranges, the primary decision is often whether to make the variable discrete or continuous. For example, the *Ruggedness* of the vehicle could be a continuous variable between 0 and 1, or a discrete variable with range $\{TinCan, Normal, Tank\}$.

[4] Some insurance companies also acquire the applicant's credit history to help in assessing risk; this provides considerably more information about socioeconomic category. Whenever using hidden variables of this kind, one must be careful that they do not inadvertently become proxies for variables such as race that may not be used in insurance decisions. Techniques for avoiding biases of this kind are described in Chapter 19.